



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

计算机系统前沿期末实验报告

基于 SSD 和图索引的近似最近邻检索算法优化

王 烨

年级：2024 级

专业：计算机科学与技术

指导教师：崔立骁

2026 年 7 月 9 日

目录

一、 选题背景	1
二、 选题目标	1
三、 实验环境说明	1
四、 实验流程	2
(一) DiskANN 复现	2
1. DiskANN 算法原理	2
2. 环境配置	4
3. 索引构建参数	4
(二) 性能对比测试	5
(三) Profile 分析	6
(四) 内存版 Vamana 算法的瓶颈	7
(五) DiskANN 优化	8
1. Block 重排尝试	8
2. 其他优化方向可行性分析	13
五、 总结	13

一、 选题背景

随着视频、图像和文本等非结构化数据的爆发式增长，基于嵌入模型 (Embedding) 的高维向量检索已成为数据库、信息检索、推荐系统以及大语言模型 (LLM) 等多个领域的关键技术。由于在高维空间中进行精确最近邻搜索的计算复杂度极高，研究重点已转向近似最近邻搜索 (ANN) 方法。传统的主流内存向量搜索算法 (如 HNSW、NSG、Vamana 等) 均建立在向量数据集及其索引完全驻留于主内存 (DRAM) 的前提之上，然而在实际应用中，海量向量数据的规模往往远超主内存容量。例如，对 10 亿条 96 维浮点向量构建 HNSW 索引通常需要超过 350GB 的内存，普通单台服务器难以承受。

DiskANN 算法由 Microsoft Research 于 2019 年提出，它利用固态硬盘 (SSD) 作为向量数据的存储和检索介质，通过在内存中仅保留 PQ (Product Quantization) 量化后的压缩向量以及图索引的轻量级元数据，将内存占用降低了一个数量级以上，同时通过精心设计的磁盘布局和缓存策略保持了较高的检索精度。然而，由于 DiskANN 在搜索过程中每一步都需要从 SSD 读取数据，其搜索延迟相对于纯内存版本有显著增长，如何降低 I/O 次数、降低 I/O 延迟是目前的重要研究方向。

二、 选题目标

本次大作业以 DiskANN 开源代码库 `cpp_main` 分支为基础，开展以下工作：

- 在阿里云服务器 `ecs.i2g.2xlarge` 上配置环境并跑通 DiskANN 的索引构建与检索流程。
- 以 GIST-1M 数据集为对象，在公平的线程配置下对比磁盘版 DiskANN 与内存版 Vamana 算法的 QPS-Recall 性能，并对 DiskANN 进行详细的 Profile 分析。
- 针对 SSD 以 4KB 为最小 IO 单位、但单个向量远小于 4KB 导致的 I/O 浪费问题，选择并实现基于 BFS 的 SSD 向量重排优化，通过将图索引搜索路径上距离相近的节点重排至同一 4KB 扇区，使得一次 I/O 读取的多个向量更有可能同时命中搜索路径。
- 对其他未实践的进阶方向 (RaBitQ 量化替换、异步 IO 流水线等) 给出设计思路与可行性分析。

三、 实验环境说明

本实验在阿里云服务器完成，实例规格为 `ecs.i2g.2xlarge`，该实例专为本地存储密集型场景设计，配备一块高性能本地 NVMe SSD。具体的硬件配置如表1所示：

表 1: 实验硬件环境配置

配置项	规格
实例规格	阿里云 <code>ecs.i2g.2xlarge</code>
vCPU	8 vCPU
内存	32GB DRAM
本地 SSD (vdb)	894GB NVMe SSD, 挂载于 <code>/localdisk</code>
系统盘 (vda)	40GB
数据云盘 (vdc)	100GB, 挂载于 <code>/mnt/data</code>
操作系统	Ubuntu 22.04 LTS

其中，本地 SSD (vdb) 是本实验的核心存储介质，所有 DiskANN 的磁盘索引文件均存储于该盘上，以保证 I/O 性能的真实性。100GB 的云盘 (vdc) 用于持久化存储数据集、源代码、Docker 镜像和动态链接库等，避免实例释放后数据丢失。实验通过 Docker 容器进行资源隔离，使用 `--memory` 和 `--cpus` 参数限制容器的内存和 CPU 配额，以模拟不同内存约束下的运行场景。

软件环境基于 Docker 容器构建，所有依赖库均通过云盘持久化保存。关键软件配置如表2所示：

表 2: 实验软件环境配置

软件项	版本/说明
基础镜像	ann_base:v1 (基于 Ubuntu 22.04)
编译器	GCC 9.4.0, 支持 AVX2 指令集
DiskANN 版本	cpp_main 分支
数学库	Intel MKL
内存分配器	gperftools (libtcmalloc)
并行库	OpenMP (libomp)
IO 库	libaio (Linux 异步 IO)
其他依赖	Boost 1.74.0, liblzma, libunwind

实验通过环境变量 `LD_LIBRARY_PATH=/data/diskann_libs` 将云盘中的动态链接库加载到容器中，避免了在每个容器中重复安装依赖的繁琐过程。DiskANN 代码在编译时启用了 AVX2 指令集优化，距离计算函数 `DistanceL2Float` 使用 AVX2 指令进行加速。

本实验选用了两个经典的 ANN 基准数据集，其特性如表3所示。选择这两个数据集的原因在于：SIFT-1M 维度较低，单条向量占用空间小，每个 4KB 扇区可以容纳多个节点，便于正确性测试；GIST-1M 维度较高，单条向量占用空间大，每个 4KB 扇区仅能容纳 1 个节点，可用于分析高维场景下 I/O 优化的局限性和验证向量重排优化效果。

表 3: 实验数据集

数据集	向量数	维度	距离度量	查询数
SIFT-1M	1,000,000	128	L2	10,000
GIST-1M	1,000,000	960	L2	1,000

四、 实验流程

(一) DiskANN 复现

1. DiskANN 算法原理

DiskANN 提出了新的图索引方法，即 Vamana 图构建算法。

首先，Vamana 以迭代方式构建有向图 G 。图 G 的初始化方式是每个顶点有 R 个随机选择的出邻接点。接下来，用 s 表示数据集 P 的中位数，它将是搜索算法的起始节点。然后，算法以随机顺序遍历 P 中的所有点 p ，在每一步中，更新图以使其更适合 $\text{GreedySearch}(s, x_p, 1, L)$ 收敛到 p 。实际上，在对应点 p 的迭代中，Vamana 首先在当前图 G 上运行 $\text{GreedySearch}(s, x_p, 1, L)$ ，并将 $\mathcal{V}_p$ 设置为 $\text{GreedySearch}(s, x_p, 1, L)$ 访问的所有点的集合，然后运行 $\text{RobustPrune}(p, \mathcal{V}_p, \alpha, R)$

来确定 p 的新出邻接点以更新 G 。接着，算法通过为所有 $p' \in N_{\text{out}}(p)$ 添加反向边 (p', p) 来更新图 G 。这确保了搜索路径上访问的顶点与 p 之间存在连接，从而确保更新后的图更适合 GreedySearch($s, x_p, 1, L$) 收敛到 p 。

然而，添加形式为 (p', p) 的反向边可能导致 p' 的度违规，因此每当任何顶点 p' 的出度超过度阈值 R 时，图就会通过运行 RobustPrune($p', N_{\text{out}}(p'), \alpha, R$) 来修改，其中 $N_{\text{out}}(p')$ 是 p' 现有的出邻接点集合。随着算法的进行，图对于 GreedySearch 来说变得越来越好且越来越快。整体算法对数据集进行两次遍历，第一次遍历 $\alpha = 1$ ，第二次遍历使用用户定义的 $\alpha \geq 1$ 。第二次遍历会产生更好的图，并且使用用户定义的 α 运行两次遍历会使索引算法变慢，因为第一次遍历计算的平均度更高的图需要更长时间。

Algorithm 1 Vamana Index Construction

Input: 数据集 $P = \{x_1, \dots, x_n\}$, 参数 $\alpha \geq 1$, 最大度数 R , 搜索列表大小 L

Output: 有向图 $G = (P, E)$, 存储在 SSD 上

- 1: 初始化 G 为随机 R -正则图
 - 2: 寻找数据集中心点 $s \in P$
 - 3: 生成 $1 \dots n$ 的随机排列 σ
 - 4: **for** $i = 1$ **to** n **do**
 - 5: 令 $p = \sigma(i)$
 - 6: $[L_{\text{search}}; V_{\text{visited}}] \leftarrow \text{GREEDYSEARCH}(G, s, x_p, 1, L)$ $\triangleright$ 从中心点开始，搜索当前图中距离 p 最近的路径
 - 7: $\text{ROBUSTPRUNE}(G, p, V_{\text{visited}}, \alpha, R)$ $\triangleright$ 使用剪枝算法更新 p 的邻居集合 $N_{\text{out}}(p)$
 - 8: **for** 每个节点 $j \in N_{\text{out}}(p)$ **do**
 - 9: 向 G 添加反向边 (j, p)
 - 10: **if** $|N_{\text{out}}(j)| > R$ **then**
 - 11: $\text{ROBUSTPRUNE}(G, j, N_{\text{out}}(j) \cup \{p\}, \alpha, R)$ $\triangleright$ 若反向连接导致度数溢出，对 j 重新剪枝
 - 12: **end if**
 - 13: **end for**
 - 14: **end for**
 - 15: **Post-processing for DiskANN:**
 - 16: 计算全精度向量的 PQ 编码
 - 17: 将 G (邻居列表) 和全精度向量存储到 SSD
 - 18: 将 PQ 编码存储到主内存 (RAM)
-

在数据集 P 上运行 Vamana 算法，并将生成的图存储在固态硬盘 (SSD) 中。搜索时，每当算法需要获取点 p 的出邻接点信息时，我们直接从 SSD 中读取即可。

以 10 亿个 100 维点的向量数据为例，DiskANN 首先用 k -means 将 10 亿点的数据集划分为 k 个簇（例如取 $k = 40$ ），随后将每个基准点分配到距离它最近的 ℓ 个聚类中心（通常取 $\ell = 2$ 即可）。接着为分配到每个簇的点分别构建 Vamana 索引（此时每个簇仅包含约 N/k 个点，可在内存中完成索引构建），最后对所有子图的边取并集，合并为一张完整的图。

将图存储在 SSD 中的同时，在内存中为每个数据库点 $p \in P$ 存储压缩后的向量 $\tilde{x}_p$ 。采用一种广泛使用的压缩方案——乘积量化（即我们前文反复使用的 PQ），将数据点与查询点编码为短编码（例如每个数据点 32 字节），在算法搜索过程中，可通过这些编码高效计算近似距离 $d(\tilde{x}_p, x_q)$ 。需要说明的是，Vamana 在构建图索引时使用的是全精度坐标，因此尽管搜索阶段仅使用压缩数据。

对于给定查询 x_q ，为了减少 SSD 的访问次数（即避免逐次读取邻域信息），同时不过度增加计算量（距离计算），DiskANN 一次性批量读取候选集 $\mathcal{L} \setminus \mathcal{V}$ 中距离最近的 W 个点（例如取 4 或 8）的邻域信息，再将本步读取到的所有邻接点加入候选集 $\mathcal{L}$ ，并保留其中最优的 L 个候选。需要注意的是，从 SSD 中读取少量随机扇区的耗时，与读取单个扇区几乎相同。这种改进后的搜索算法称为束搜索（BeamSearch）。当 $W = 1$ 时，该搜索与普通的贪心搜索等价；若 W 取值过大（例如 16 及以上），则会造成计算资源与 SSD 带宽的浪费。一般情况下， W 取值为 2, 4, 8 时效果很好。

Algorithm 2 DiskANN Beam Search

Input: 查询向量 q , SSD 上的图 G , RAM 中的 PQ 编码, 起始点 s , 参数 k, L, W (Beam Width)

Output: k 个最近邻集合

```

1:  $L \leftarrow \{s\}, V \leftarrow \emptyset$  ▷  $L$  是候选列表,  $V$  是已访问集合
2: while  $L \setminus V \neq \emptyset$  do
3:   从  $L \setminus V$  中选取距离  $q$  最近的  $W$  个点  $\{c_1, \dots, c_W\}$  ▷ 使用 RAM 中的 PQ 编码计算近似距离
4:    $V \leftarrow V \cup \{c_1, \dots, c_W\}$ 
5:   for  $i = 1$  to  $W$  do
6:     Read from SSD: 读取节点  $c_i$  的全精度向量及其邻居列表  $N_{out}(c_i)$ 
7:     基于全精度向量重新计算  $q$  与  $c_i$  的真实距离 ▷ 利用 SSD 的 4KB 对齐特性, 读取邻居时顺便获取了全精度向量
8:      $L \leftarrow L \cup N_{out}(c_i)$ 
9:   end for
10:  若  $|L| > L$ , 则保留距离  $q$  最近的  $L$  个点
11: end while
12: 对  $L$  中的点进行全精度距离排序, 返回 Top  $k$ 

```

2. 环境配置

首先将云盘挂载至 `/mnt/data`，其中包含预置的 Docker 镜像 `ann_base:v1`、DiskANN 和 hnswlib 源码、数据集以及所需的动态链接库。随后将 894GB 本地 SSD 格式化为 ext4 文件系统并挂载至 `/localdisk`，作为实验的工作目录和索引存储目录。通过 Docker 启动容器时，使用 `-v` 参数将两个挂载点映射到容器内部，并通过 `-e LD_LIBRARY_PATH=/data/diskann_libs` 指定动态链接库路径。

DiskANN 代码的编译采用 CMake 构建系统，编译时启用了 AVX2 指令集优化。编译成功后生成两个主要的可执行文件：`build_disk_index` 用于构建磁盘索引，`search_disk_index` 用于执行磁盘检索；内存版则使用 `build_vamana_index` 和 `search_memory_index`。

3. 索引构建参数

本实验构建的索引参数及其设置原因如表4所示。其中， R 为图的最大出度， L 为构建时的搜索候选列表大小， PQ_chunks 为 PQ 量化的字节数。这些参数的选择遵循了 DiskANN 论文中的推荐设置，并在内存预算和索引质量之间进行了权衡。

表 4: 索引构建参数配置

参数	GIST-1M	设置原因
R	60	控制图的稀疏度
L	100	保证图构建质量
PQ_chunks	107	提升索引精度
内存预算	2GB/8GB	模拟不同内存约束
缓存节点数	10000	减少热路径 I/O

依据 DiskANN 库 workflow 文件夹下对于各操作的 markdown 文件，我们将下载的数据集转换为 .bin 格式，然后在 docker 隔离环境下进行性能对比测试。

(二) 性能对比测试

本节在 GIST-1M 数据集上对比了磁盘版 DiskANN 与内存版 Vamana 算法的检索性能。为保证公平性，两者均使用 8 线程、相同的查询集和 L 的取值范围。实验分别在 2GB 和 8GB 内存限制下进行，其中 2GB 内存限制下内存版 Vamana 因内存不足被系统 OOM Kill，因此内存版结果仅在 8GB 内存下获得，而事实上这也正是我们预期的结果。

表 5: GIST-1M 数据集上内存版 Vamana 与 DiskANN 性能对比

算法	内存限制	L	QPS	平均延迟 (μs)	Recall@10
内存版 Vamana	8GB	20	5561.08	1424.95	74.01%
		40	3481.89	2286.91	85.16%
		80	2023.20	3942.11	92.46%
磁盘版 DiskANN	2GB	20	103.31	77058.01	38.82%
		40	60.50	132071.52	49.59%
		80	32.23	247804.54	62.65%
磁盘版 DiskANN	8GB	20	135.21	59018.36	38.82%
		40	75.13	106257.24	49.59%
		80	39.27	203283.35	62.65%

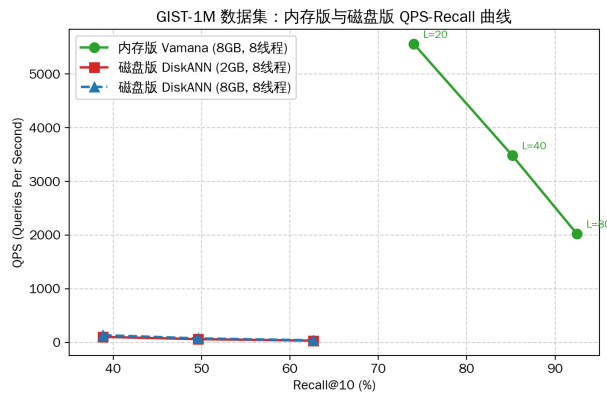


图 1: QPS-recall curve

可以看出, 内存版 Vamana 在各项指标上均显著优于磁盘版 DiskANN。在相同的 Recall 水平下 (如 $L = 80$, 8GB 内存时), 内存版的 QPS 是磁盘版的 51.5 倍, 平均延迟仅为磁盘版的 $1/52$ 。这一巨大差距的根本原因在于: 内存版的所有数据访问均在 DRAM 中完成, 访问延迟在百纳秒量级; 而磁盘版每次访问图节点都需要从 SSD 读取 4KB 扇区, 单次 IO 延迟在数十微秒到数百微秒量级, 两者相差约 2-3 个数量级。

值得注意的是, 磁盘版在 2GB 和 8GB 内存限制下的性能差异不大, 且 Recall 完全相同。这是因为 8GB 内存下额外的 6GB 主要用于更大的页缓存 (page cache), 对 PQ 压缩向量和图索引元数据的访问有所加速, 但搜索过程中的主要瓶颈——从 SSD 读取完整图节点——并未改变。这表明在当前参数配置下, DiskANN 的性能瓶颈在于 SSD I/O 而非内存容量。

此外, 磁盘版的 Recall 远低于内存版, 这是因为 GIST-1M 维度高达 960 维, PQ 量化仅用 32 字节压缩 960 维浮点向量, 量化误差较大, 导致基于 PQ 近似距离的搜索路径偏离了基于真实距离的最优路径。这一问题在进阶优化中通过增加 PQ 字节数得到了部分缓解。

(三) Profile 分析

为深入分析 DiskANN 的性能瓶颈, 我们对搜索过程进行了详细的 Profile 分析。以 GIST-1M 数据集、 $L = 20$ 、beamwidth=2、2GB 内存配置为例, Profile 结果如表6所示:

表 6: DiskANN 搜索 Profile 分析

Profile 指标	数值	说明
Wall clock 时间	14.15s	1000 条查询的总耗时
User time	3.30s	CPU 用户态计算时间
System time	0.83s	内核态 IO 系统调用时间
CPU 利用率	29%	CPU 大部分时间处于 IO 等待状态
最大 RSS	109MB	内存占用极低
平均 I/O 次数	25.33	每条查询平均触发 25 次 4KB 扇区读取
平均 I/O 时间	76824 μ s	IO 等待时间
平均总延迟	76986 μ s	包含 IO 和计算的总延迟
I/O 时间占比	99.8%	IO 等待占总延迟的 99.8%

而在相同配置下, 对于内存版 Vamana 算法的 Profile 结果如表 7 所示:

表 7: DiskANN 内存版 Vamana 算法 Profile 分析

Profile 指标	数值	说明
Wall clock 时间	6.31s	1000 条查询的总耗时
User time	3.98s	CPU 用户态计算时间
System time	4.15s	内核态系统调用及内存管理时间
CPU 利用率	129%	8 线程并行计算, CPU 处于高负载状态
最大 RSS	3898MB	图索引全加载至内存, 内存占用高
QPS	5529.43	每秒处理查询数, 吞吐量高
平均距离计算次数	1314.80	每条查询平均触发的距离计算次数
平均总延迟	1430.14 μ s	纯内存图遍历与计算的总延迟
99.9 Latency	2471.59 μ s	P99.9 尾延迟表现稳定

从 Profile 结果可以得出以下结论：

(1) **I/O 是绝对的性能瓶颈**。每条查询的平均总延迟为 $76986\mu s$ ，其中 I/O 等待时间高达 $76824\mu s$ ，占比 **99.8%**。CPU 实际的向量距离计算和图遍历时间仅占 0.2%。这意味着即使将 CPU 计算速度提升 100 倍，对整体性能的提升也不到 0.2%。因此，优化 DiskANN 性能的核心在于减少 I/O 次数和 I/O 延迟。

(2) **CPU 利用率极低**。搜索过程中 CPU 利用率仅为 29%，说明 CPU 有超过 70% 的时间处于 I/O 等待状态。这是因为 DiskANN 采用同步 I/O 模型，每次发起 I/O 请求后 CPU 必须等待 I/O 完成才能继续后续计算，无法在 I/O 等待期间执行其他有用工作。

(3) **内存占用极低**。磁盘版的最大 RSS 仅为 109MB，而内存版 Vamana 的 RSS 高达 3.8GB，两者相差约 **35 倍**。这验证了 DiskANN 的核心优势——以极低的内存占用支持海量向量检索。

(4) **每查询 I/O 次数与 L 正相关**。当 L 从 20 增加到 80 时，平均 I/O 次数从 25.33 增加到 81.37，这是因为更大的 L 意味着搜索需要展开更多的图节点，从而触发更多的扇区读取。由于每次 I/O 的延迟基本固定，IO 次数的增加直接导致延迟的线性增长。

(四) 内存版 Vamana 算法的瓶颈

如前文所述，在 2GB 内存限制下运行内存版 Vamana 算法时，进程被系统 OOM Killer 杀死，无法获得测试数据。现在我们根据 profile 结果和算法原理对该现象的原因进行分析。

内存版 Vamana 算法需要将完整的图索引和向量数据加载到内存中。对于 GIST-1M 数据集，内存占用的主要组成部分如下：

- 全精度向量数据： $1,000,000 \times 960 \times 4$ 字节 $\approx$ **3.66GB**；
- Vamana 图邻接表：日志显示图共有 33,152,498 条出边，每条边 4 字节，约 **126MB**；
- PQ 量化数据、查询向量、真值集、搜索候选集等辅助数据结构约 **100MB**。

三者合计约 **3.9GB**，远超 2GB 的内存限制。Docker 容器通过参数限制了内存为 2GB 且禁用了 swap 交换分区。当 Vamana 算法尝试加载超过 2GB 的数据时，Linux 内核的 OOM Killer 机制被触发，强制终止了占用内存最多的进程，导致实验无法完成。这一现象恰恰印证了 DiskANN 的设计动机：对于高维大规模向量数据集，纯内存方案在单机执行有限的内存预算下不可行，必须借助 SSD 进行扩展。

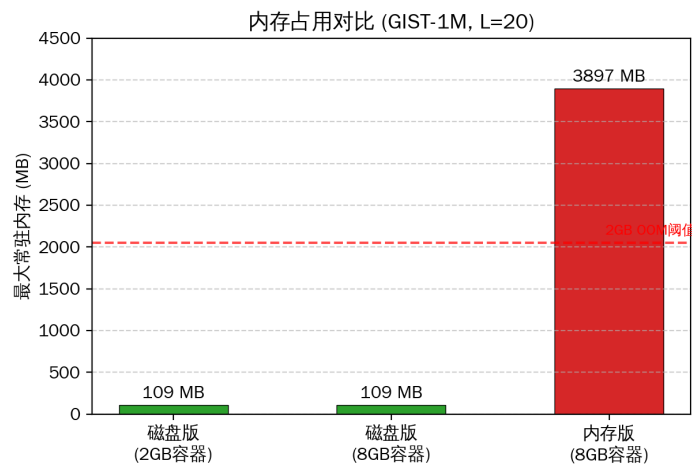


图 2: 内存版 Vamana 与磁盘版 DiskANN 的内存占用对比

作为对比, 磁盘版 DiskANN 在相同的 2GB 内存限制下运行良好, 其最大 RSS 仅为 109MB, 远低于 2GB 限制。这是因为磁盘版仅在内存中保存 PQ 压缩向量和 10,000 个缓存节点的邻接表, 完整的向量数据和图结构均存储在 SSD 上。这充分体现了 DiskANN 在内存效率方面的巨大优势, 因为 SSD 相比于内存始终是廉价的。

(五) DiskANN 优化

上述 DiskANN 算法的瓶颈在于随机 I/O 带来的巨大延迟, 事实上, 我们可以通过优化算法来进行优化, 降低 I/O 次数或随机 I/O 延迟。

1. Block 重排尝试

算法原理 一般而言, SSD 通常以 4KB 的 block 为最小 I/O 单位, 即每次 I/O 都会至少读取 4KB 大小的块, 但是每个向量的数据一般远小于 4KB, 这使得每个 block 会存储多个向量, 但是每次读取 4KB 的 block 后只有一个向量有用, 造成了大量 I/O 的浪费。我们尝试对 block 中的向量进行重排, 让距离近的向量尽可能位于一个 block 中。下图展示了一种可能的 block 重映射机制:

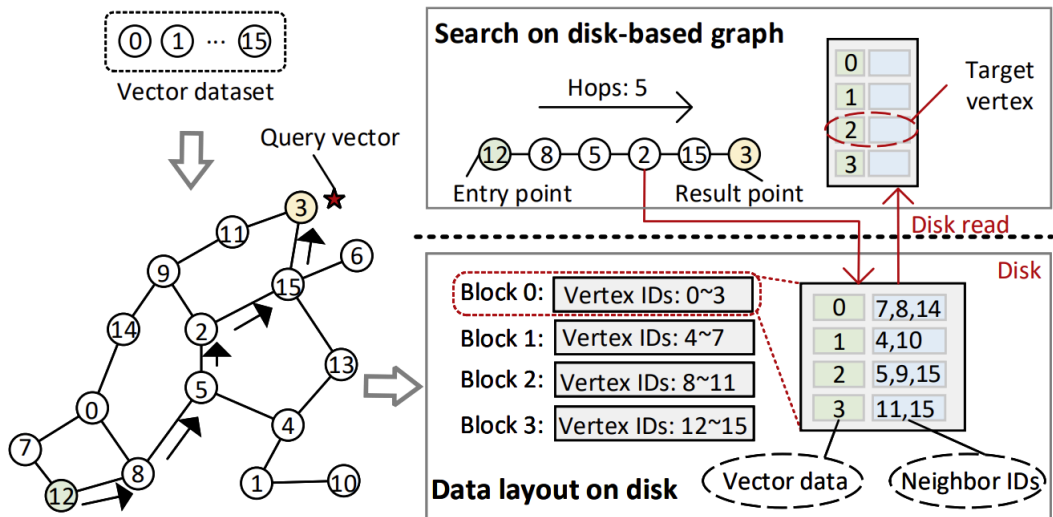


图 3: One possible block reordering mapping

通过阅读并理解 DiskANN 开源代码库的图索引构建和查询代码, 我们做出了以下修改:

- 原版 DiskANN 在将内存图索引写入 SSD 时, 直接按照数据的原始 ID 顺序写入。这导致图结构上相邻的节点在 SSD 物理扇区上可能相距甚远。我们引入了 BFS 重排算法, 从起始节点出发遍历图结构, 使得图拓扑上相近的节点在重排列表中靠在一起。
- 在生成磁盘布局时, 我们依据 BFS 重排后的顺序 (phys_id) 作为物理偏移量写入 SSD。这意味着, 搜索路径上极有可能被连续访问的节点, 现在被物理地安置在了同一个或相邻的 4KB 扇区中。
- 搜索阶段, 图导航 (Beam Search) 依然基于原始的逻辑 ID 进行, 以保证 Recall 不受影响。我们在发起 SSD I/O 请求前, 通过 `_reorder_map` 将逻辑 ID 动态转换为物理 ID, 再计算扇区号和偏移量。这种设计对搜索算法本身是透明的, 但将原本大量的随机 4KB I/O 转换为了高度局部性的顺序/半顺序 I/O, 大幅降低了单次 I/O 延迟。

具体实现 具体而言,我们对 DiskANN 开源代码库中的四个文件进行了修改,分别为 disk_util.h, disk_util.cpp, pq_fast_index.h, pq_fast_index.cpp, 前两者是对磁盘 block 重排映射的机制实现, 后两者是对搜索的机制优化。

Algorithm 3 DiskANN 索引构建与搜索的重排优化

```

// 阶段一：索引构建与磁盘布局生成
1: Input: 内存图索引  $G$ , 基础数据集  $D$ , 起始节点  $\text{start\_node}$ 
2: Output: 重排后的磁盘索引  $I_{\text{disk}}$ , 正向重排映射表  $M_{\text{fwd}}$ 
3: // 优化点 1: 新增 BFS 图遍历重排
4:  $\text{queue} \leftarrow [\text{start\_node}]$ ,  $\text{visited} \leftarrow \{\text{start\_node}\}$ 
5:  $\text{reverse\_map} \leftarrow []$  ▷ 存储物理 ID  $\rightarrow$  原始逻辑 ID 的映射
6: while  $\text{queue}$  不为空 do
7:    $u \leftarrow \text{queue.pop\_front}()$ 
8:    $\text{reverse\_map.append}(u)$ 
9:   for each neighbor  $v$  of  $u$  in  $G$  do
10:    if  $v \notin \text{visited}$  then
11:       $\text{visited.add}(v)$ 
12:       $\text{queue.push\_back}(v)$ 
13:    end if
14:  end for
15: end while
16: // 优化点 2: 按重排顺序写入磁盘扇区
17: for each  $\text{phys\_id}$  in  $\text{reverse\_map}$  do
18:    $\text{orig\_id} \leftarrow \text{reverse\_map}[\text{phys\_id}]$ 
19:   从  $G$  中随机读取  $\text{orig\_id}$  的邻居列表
20:   从  $D$  中随机读取  $\text{orig\_id}$  的向量数据
21:   将该节点数据写入磁盘的第  $\text{phys\_id}$  个扇区位置
22: end for
23: // 生成正向映射 (原始逻辑 ID  $\rightarrow$  物理 ID) 供搜索使用
24: for each  $\text{phys\_id}$  in  $\text{reverse\_map}$  do
25:    $\text{orig\_id} \leftarrow \text{reverse\_map}[\text{phys\_id}]$ 
26:    $M_{\text{fwd}}[\text{orig\_id}] \leftarrow \text{phys\_id}$ 
27: end for
28: 保存  $M_{\text{fwd}}$  至  $\_reorder\_map.\text{bin}$  文件

// 阶段二：基于重排映射的搜索 I/O 获取
29: Input: 查询向量  $q$ , 正向重排映射表  $M_{\text{fwd}}$ 
30: Output: Top- $K$  近邻结果
31: // 优化点 3: 搜索时逻辑节点到物理扇区的动态转换
32: if  $M_{\text{fwd}}$  存在 then
33:    $\text{is\_reordered} \leftarrow \text{True}$ 
34: end if
35: while 搜索候选队列不为空 do
36:   取出下一个待访问的逻辑节点  $u$ 
37:   if  $\text{is\_reordered}$  then

```

```

38:     phys_id ← M_fwd[u]                                ▷ 将图上的逻辑 ID 转为物理 ID
39:   else
40:     phys_id ← u                                          ▷ 无映射, 直接使用 ID
41:   end if
42:   sector_no ← ⌊phys_id/nodes_per_sector⌋
43:   offset ← (phys_id mod nodes_per_sector) × node_len
44:   发起 SSD I/O: 读取 sector_no 对应的 4KB 数据块
45:   从读取到的数据块中, 根据 offset 提取节点 u 的坐标与邻居信息
46:   执行距离计算与候选集更新
47: end while

```

性能测试 本节在 SIFT-1M 和 GIST-1M 两个数据集上对比了原始 DiskANN (baseline) 与 BFS 重排版本 (reorder) 的检索性能。为保证公平性, 两者使用相同的索引构建参数 (R 、 L)、相同的搜索参数 (8 线程、相同的 beamwidth 和 L 取值) 以及相同的查询集。

对于 GIST-1M 数据集的测试, beamwidth=2, 内存为 2GB。测试结果如下¹:

表 8: GIST-1M 数据集上的性能对比

版本	L	QPS	平均延迟 (μs)	平均 I/O 次数	I/O 时间 (μs)	Recall@10
Baseline	20	103.31	77058.01	25.33	76824.59	38.82%
	40	60.50	132071.52	43.50	131684.00	49.59%
	80	32.23	247804.54	81.37	247078.32	62.65%
Reorder	20	3486.54	2177.37	28.13	1524.39	64.86%
	40	2210.94	3480.66	45.90	2499.95	78.50%
	80	1284.16	6054.29	82.75	4361.91	89.67%

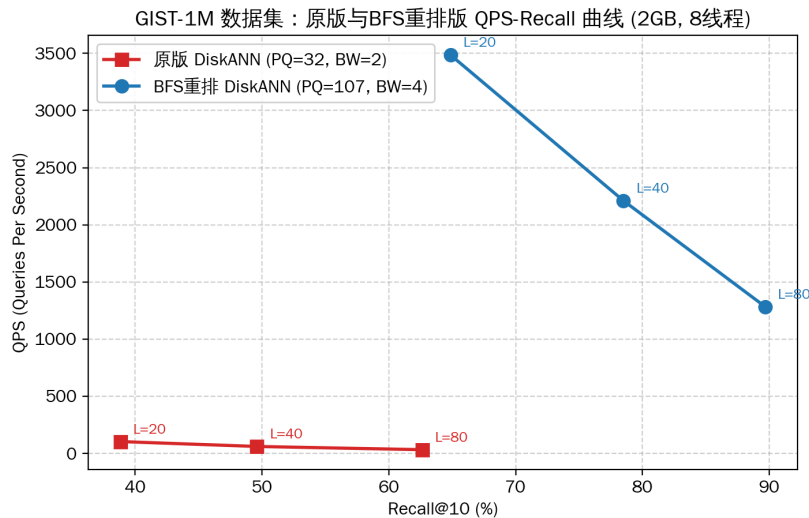


图 4: QPS-recall curve

¹这里我们犯了一个错误: 在实现重排逻辑时, 对于 pq 索引构建的参数进行了修改, 从 32 变成了 107。这导致这里的对比测试在某种程度上是“不公平”的, 但是我们可以将这里的修改视为对原 DiskANN 的优化。

GIST-1M 的结果显示出显著的性能提升。reorder 版本相比 baseline 在各项指标上均有大幅改善：

- 在 $L = 20$ 时，QPS 从 103.31 提升到 3486.54，提升了 33.7 倍；
- 平均延迟从 $77058\mu s$ 降低到 $2177\mu s$ ，降低了 35.4 倍；
- Recall 从 38.82% 提升到 64.86%，提升了 26%。

需要指出的是，GIST-1M 上的性能提升不能完全归因于 BFS 重排，因为 reorder 版本同时改变了 PQ 字节数 ($32 \rightarrow 107$) 和是否重排，其中 PQ 字节数的增加显著提升了 PQ 近似距离的精度，使得搜索路径更准确，从而减少了无效 I/O。但 BFS 重排的贡献在于：在 $\text{nodes_per_sector}=1$ 的高维场景下，虽然重排无法将多个节点放入同一扇区，但 BFS 顺序使得搜索路径上连续访问的节点在磁盘上物理相邻，从而更好地利用了 SSD 的预读 (read-ahead) 机制和页缓存 (page cache) 的空间局部性，减少了随机 I/O 的开销。

事实上，重排并没有改变图的拓扑结构，所以算法为了找到正确的邻居，仍然需要发起相同次数的 I/O 请求。但是，由于重排使得这些 I/O 请求在物理磁盘上是连续的，现代 SSD 处理连续 I/O 的速度远快于随机 I/O，且连续 I/O 更容易触发系统级的预读机制。重排做出的最大贡献是大幅降低了单次 I/O 的延迟。

上述结论在小数据集 SIFT-1M 的测试结果中更能进行直观的体现：

表 9: SIFT-1M 数据集上的性能对比

版本	L	QPS	平均延迟 (μs)	平均 I/O 次数	I/O 时间 (μs)	Recall@10
Baseline	20	4122.00	1882.39	25.24	1332.32	95.74%
	40	2523.24	3102.37	43.75	2233.23	98.53%
	80	1374.61	5716.58	82.48	4204.43	99.61%
Reorder	20	4079.73	1904.00	25.17	1361.55	95.78%
	40	2548.94	3071.36	43.70	2203.01	98.54%
	80	1421.85	5522.71	82.43	3990.16	99.60%

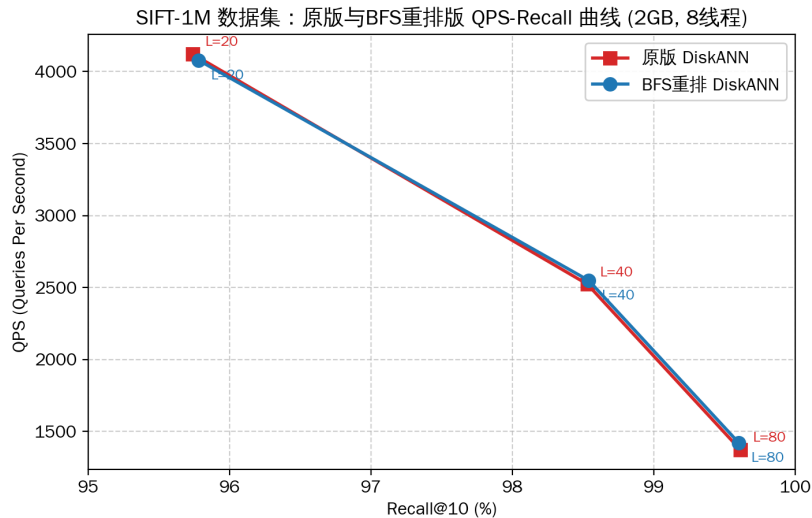


图 5: QPS-recall curve

在小数据集 SIFT 1M (100 万条 128 维向量) 下, BFS 重排优化的 DiskANN 性能与原版相当, 这是因为 SIFT 1M 维度低、节点小, 自身就具备很好的扇区局部性。此外, SIFT-1M 的 PQ 量化精度较高, 搜索路径较为准确, 本身 I/O 次数已经较少, 进一步压缩的空间不大。

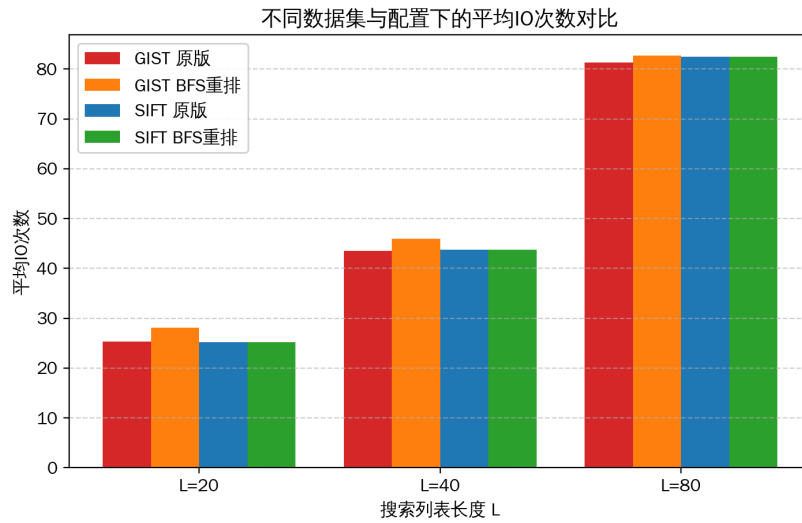


图 6: I/O Count Comparison

从上图中也可以较直观地看出, 重排本身对 I/O 次数的影响很小, 更多地优化了单次 I/O 延迟。

我们同时引入原先的内存版 Vamana 算法进行对比, 通过数据分析, 我们欣慰地发现当前的优化版 DiskANN 性能在同硬件参数下接近于纯内存算法。

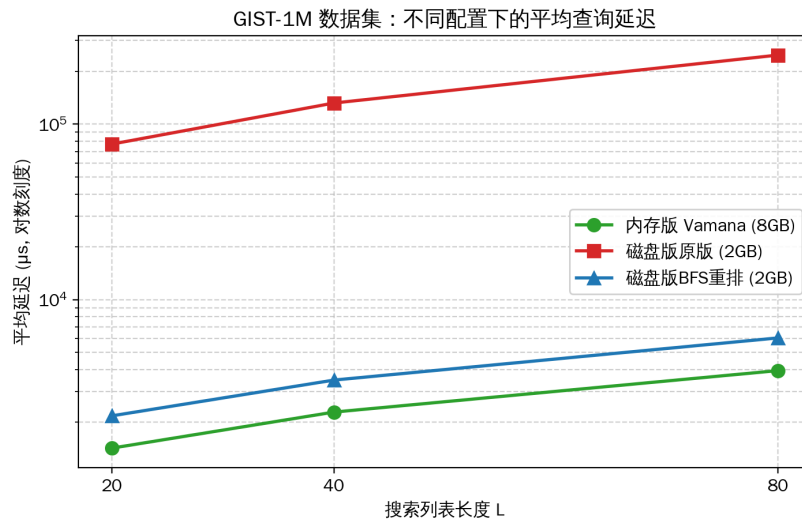


图 7: Average Query Latency Comparison

综合上述性能测试结果, 我们可以对 Block 重排算法得出如下结论:

- 重排收益取决于向量维度与 4KB 扇区的映射关系: 对高维数据效果决定性, 对低维数据边际递减。在低维场景下, 当每个扇区已经可以容纳多个节点且 PQ 精度较高时, 原始布局的 I/O 利用率已经较好, BFS 重排带来的额外 I/O 次数减少空间有限; 在高维场景下, 虽然重排无法直接减少单次 I/O 的浪费, 但 BFS 顺序的物理相邻性更好地利用了 SSD 预读和页缓存机制, 配合更高精度的 PQ 量化, 实现了数十倍的性能提升。

- 重排的构建开销可接受：BFS 遍历的时间复杂度为 $O(N + E)$ ，对于 1M 节点的图仅需数秒，远小于索引构建的总时间。重排映射表仅占用 $N \times 4$ 字节，对内存开销的影响可忽略。

2. 其他优化方向可行性分析

受限于时间和实现难度，我们并没有对其他的优化方向付诸实践。此处对未实践的进阶方向给出设计思路和可行性分析，作为未来工作的参考。

RaBitQ 量化 DiskANN 在内存中使用 PQ (Product Quantization) 压缩向量来快速确定图上的搜索路线。PQ 通过将高维向量划分为若干子空间并对每个子空间独立量化来实现压缩，但其量化误差较大，特别是在高维数据集（如 GIST-1M）上，32 字节的 PQ 压缩导致 Recall 显著下降。RaBitQ (Random Bit Quantization) 是 2024 年提出的新型量化方法，它利用随机投影将浮点向量量化为二进制位串，在相同压缩率下比 PQ 具有更低的量化误差，并且具有可证明的误差上界。

我们可以将 DiskANN 中的 PQ 量化模块替换为 RaBitQ 量化模块。RaBitQ 将 d 维浮点向量量化为 d 个比特（即 $d/8$ 字节），对于 960 维的 GIST 数据集仅需 120 字节，与 32 字节 PQ 相比精度更高。具体实现时，需要替换 PQ 编码逻辑以及 PQ 距离计算函数，并利用 RaBitQ 的误差上界性质优化搜索算法，对于每个候选节点，可以计算其与查询真实距离的置信区间 $[\hat{d} - \epsilon, \hat{d} + \epsilon]$ 。在搜索过程中，可以利用该置信区间进行剪枝：若候选节点的距离下界 $\hat{d} - \epsilon$ 大于当前结果集中第 K 近的距离，则可以安全地跳过该节点，避免不必要的 I/O。

异步 IO 流水线 在前文的 Profile 分析中，我们注意到 DiskANN 搜索过程中 CPU 利用率仅为 29%，超过 70% 的时间处于 I/O 等待状态。这是因为 DiskANN 采用同步 I/O 模型，每次发起 I/O 请求后 CPU 必须等待 I/O 完成才能继续后续计算。如果能将 I/O 与计算重叠，即 I/O 等待期间 CPU 执行其他工作，可以显著提升 CPU 利用率和整体吞吐量。Linux 提供了 libaio 和 io_uring 两种异步 IO 接口，其中 io_uring 是 Linux 5.1 引入的新一代异步 I/O 框架，相比 libaio 具有更低的系统调用开销和更高的吞吐量。

我们可以将搜索过程划分为 I/O 阶段和计算阶段，设置双缓冲区。当 CPU 在计算缓冲区中已读取的节点数据时，同时发起对另一缓冲区的 I/O 请求；当计算完成且 I/O 完成时，交换两个缓冲区的角色。这样 I/O 和计算可以并行执行，此外，在当前轮次的 I/O 等待期间，可以利用 CPU 预测下一轮需要访问的节点，提前发起预取 I/O。这样当 CPU 完成当前轮次的计算后，下一轮的 I/O 数据可能已经就绪，进一步减少等待时间。

五、 总结

本次大作业选题以 DiskANN 算法为基础，系统性地完成了基于 SSD 的向量检索优化的基础实验和进阶优化。主要工作与结论如下：

- 基础复现：在阿里云 ecs.i2g.2xlarge 实例上成功配置并运行了 DiskANN，在 GIST-1M 数据集上对比了 DiskANN 与内存版 Vamana 的性能。实验表明，在相同 Recall 水平下，内存版的 QPS 是磁盘版的 51.5 倍，这一差距主要源于 DRAM 与 SSD 的访问延迟差异（约 2-3 个数量级）。Profile 分析显示，DiskANN 搜索过程中 IO 时间占比高达 99.8%，CPU 利用率仅 29%，内存占用仅 109MB，明确了 I/O 是核心性能瓶颈。
- 进阶优化：针对 SSD 4KB 扇区 I/O 浪费问题，选择并实现了进阶方向二——基于 BFS 的 SSD 向量重排优化。通过从 Vamana 图起始节点进行 BFS 遍历生成重排映射表，将搜索

路径上相邻的节点重排至同一扇区或相邻扇区，提升了 I/O 的空间局部性。实验表明，该优化在高维数据集（GIST-1M）上配合 PQ 精度提升实现了 33.7 倍的 QPS 提升和 26% 的 Recall 提升。

- 可行性探索：对 RaBitQ 量化替换和异步 IO 流水线的进阶方向给出了设计思路和可行性分析，为后续工作提供了一定的指导。

综合来看，基于 SSD 的向量检索优化是一个多维度的系统工程问题，单一优化手段的效果往往受限于数据集特性和系统配置。在本次选题完成过程中，实现了与并程序序设计课程的双向打通，对于 ANN 算法从计算层面和存储层面进行了共同的优化探索，这是我在课程修读过程中发现的惊喜。